Apache CloudStack Marvin – FizzBuzz API Test Report

Prepared by:	Raveen
Testing Tool:	Apache CloudStack Marvin
API Name:	fizzBuzz
Environment:	CloudStack QA environment

1. Introduction

The purpose of this assignment was to test a custom CloudStack API called **fizzBuzz**. Python and Apache CloudStack Marvin were utilized to send API requests and verify whether the API returned the correct answers. The API accepts an integer called `number` and is expected to return the following results based on specific conditions:

Condition	Expected Answer	Example
Number divisible by 3	**fizz**	Input 3 → fizz
Number divisible by 5	**buzz**	Input 5 → buzz
Number divisible by both 3 and 5	**fizzbuzz**	Input 15 → fizzbuzz

2. Testing Approach

A Python test file named `test_fizzbuzz_final.py` was developed using Apache CloudStack Marvin to interface directly with the CloudStack API. A comprehensive suite of 9 test methods was built to cover various edge cases and specifications:

- Number divisible by 3.
- Number divisible by 5.
- Number divisible by both 3 and 5.
- Zero as input.
- Missing required input.
- Other multiples of 3.
- Other multiples of 5.

- Negative numbers.
- Repeating the same request to check consistent results.

3. Test Execution Results

All 9 test methods were executed within the Marvin framework. The suite resulted in **5 Passes** and **4 Failures**.

Test Case	Result
Divisible by 3 (3)	**PASS**
Divisible by 5 (5)	**PASS**
Divisible by both (15)	**PASS**
Zero (0)	**FAIL**
Missing number	**PASS**
Other multiples of 3	**FAIL**
Other multiples of 5	**FAIL**
Negative numbers	**FAIL**
Repeatability	**PASS**

4. Defects Identified

Defect 1: Incorrect response for zero

Input: 0 | **Expected:** fizzbuzz | **Actual:** buzzinga | **FAIL**
Explanation: Zero is divisible by both 3 and 5. The API should return fizzbuzz, but it returned buzzinga.

Defect 2: Incorrect response for a number divisible by both

Input: 30 | **Expected:** fizzbuzz | **Actual:** buzzinga | **FAIL**
Explanation: Thirty is divisible by both 3 and 5. The API returned an unexpected answer.

Defect 3: Extra whitespace in the response

Input: 20 | **Expected:** buzz | **Actual:** buzz (with trailing space) | **FAIL**
Explanation: The API returned an extra space after the word buzz. The exact string comparison therefore failed.

Defect 4: Incorrect response for negative numbers

Input: -3 | **Expected:** fizz | **Actual:** lulz | **FAIL**
Explanation: The assignment describes the input as an integer and does not exclude negative integers. The API should return fizz for -3, but it returned lulz.

5. What Worked Correctly?

The following functional requirements successfully passed verification:

- Input 3 correctly returned **fizz**.
- Input 5 correctly returned **buzz**.
- Input 15 correctly returned **fizzbuzz**.
- Missing required inputs were correctly rejected by the command structure.
- Repeated requests with an input of 15 consistently returned the same response, verifying basic reliability.

6. Assumptions

- Zero is mathematically treated as an integer divisible by both 3 and 5.
- Negative integers are included since the specification does not explicitly exclude them.
- The `number` parameter is strictly required as indicated by the generated API command class.
- The API response payload field is named `answer`.
- Exact string matching is strictly applied, making extra whitespaces structurally invalid.

7. Conclusion

A comprehensive functional suite of 9 tests was successfully created and executed via Apache CloudStack Marvin. While basic boundary operations function as intended, four critical failures were identified involving zero, alternate common multiples, trailing whitespaces, and negative integer handling.

Final Conclusion: The `fizzBuzz` API implementation fails to fully meet the requested specification. It is highly recommended that the development team reviews these test findings to implement the required alignment fixes.